

Banknote Authentication

A Data Mining Report

Prepared By

Salma Maher

Supervised By

Dr.Rasha

FACULTY OF ENGINEERING SOFTWARE DEPARTMENT



Table of Contents

Task 1:

1. Introduction	4
2. Dataset Overview	4
3. Data Preprocessing and Cleaning Techniques	6
3.1 Handling Missing Values and Outliers	6
3.2 Normalization and Feature Scaling	6
3.3 Specific Techniques for Image and Signal Data	6
3.4 Data Balancing	7
4. Data Visualization Tools for Banknote Datasets	7
4.1 Exploring Feature Distributions and Relationships	7
5. Analysis Platforms and Software	8
5.1 Python Ecosystem	8
5.2 MATLAB and its Toolboxes	8
6. Reference.....	9

Task 2:

7. Introduction	10
a. Overview of the BankNote Authentication dataset	
8. Importing Libraries	10
9. Loading and Initial Data Exploration	11
a. Dataset shape and features	
b. Data types and class distribution	
c. Statistical summaries	
10. Missing Values Detection	12
11. Outliers Visualization	13
a. Boxplots for outlier detection	
b. Impact of outliers on analysis	
12. Outliers Removal	14
a. IQR method for outlier handling	
b. Dataset size before and after removal	
13. Boxplot After Outlier Removal	15
a. Validation of outlier removal success	
14. Feature Scaling	17
a. Standardization using StandardScaler	

15. Summary	18
a. Key steps: exploration, outlier handling, scaling, and splitting	

Task 3:

16. Library Imports	19
17. Train-Test Split	20
18. Model Training & Prediction	21
A. Decision Tree Classifier	21
B. Naive Bayes	22
C. K-Nearest Neighbors (KNN)	23
D. Support Vector Machine (SVM)	24
E. Neural Network (Keras)	25
19. Comparison: Accuracy & Confusion Matrix Heatmaps	26
20. Summary	28
A. Performance Summary	28
B. Confusion Matrix Insights	28
C. Best Performing Model(s)	29

Task 4:

21. Edit On Task 3	30
22. Evaluation Methods Implemented	31
A. Holdout Method (Train-Test Split)	31
B. Cross-Validation Methods	32
i. K-Fold CV	33
ii. Stratified K-Fold CV	34
iii. Leave-One-Out CV (LOOCV)	34
C. Bootstrap Method	35
D. ROC Curve Analysis	36
23. Model Initialization & Hyperparameter Tuning	41
A. Model Selection	41
B. Hyperparameter Optimization	41
24. Neural Network Implementation	42
A. Architecture	43
B. Training Process	43

25. Model Comparison & Results	44
A. Performance Metrics	44
B. Visualization Analysis	44
26. Analysis & Recommendations	46
27. Edits in Project.....	47-58

1. Introduction

The economy struggles with counterfeit banknotes which yield financial losses and introduces security vulnerabilities. Manual inspection of banknotes through human checks requires too much time and produces inconsistent result accuracy. Technology specifically machine learning operates to speed up and enhance the accuracy of this identification process. The following report examines how Wavelet Transform effectively retrieves critical features from banknote images by analyzing variance and skewness and curtosis together with entropy. The analysis will discuss how several machine learning models including SVM, Neural Networks and Random Forest can detect real or duplicate banknotes with excellent precision.

2. Dataset Overview

For the Banknote Authentication dataset, the data to be extracted and analyzed typically include the following key components .[1]

Features (Independent Variables):

The features stem from banknote pictures that resulted from Wavelet Transform processes. They are continuous numerical values:

- **Variance:** Measures the variance of the Wavelet Transformed image.
- **Skewness:** Measures the asymmetry of the Wavelet Transformed image.
- **Curtosis:** Measures the "tailedness" of the Wavelet Transformed image.
- **Entropy:** Measures the randomness or complexity of the image.

Target (Dependent Variable):

The class variable demonstrates whether a banknote represents a genuine or forged piece:

- **0:** Genuine banknote
- **1:** Forged banknote

Metadata:

Dataset Information:

- Number of instances: 1,372
- Number of features: 4.
- Missing values: Zero-

Variable Details:

The dataset contains the following variables[3]:

<i>Variable Name</i>	Role	Description	Data Type	Missing Values
<i>variance</i>	Feature	Variance of Wavelet Transformed image	Continuous	No
<i>skewness</i>	Feature	Skewness of Wavelet Transformed image	Continuous	No
<i>curtosis</i>	Feature	Curtosis of Wavelet Transformed image	Continuous	No
<i>entropy</i>	Feature	Entropy of the image	Continuous	No
<i>class</i>	Target	Class label (0 = genuine, 1 = forged)	Integer	No

3. Data Preprocessing and Cleaning Technique

Before analyzing any dataset, it's crucial to clean it to ensure accuracy and consistency. Data cleaning is the task of detecting or repairing corrupted, duplicate, incomplete, inaccurate, or noisy records from the dataset, to improve the quality of the dataset. Because of the importance of data quality, there is an increasing interest in developing data-cleaning approaches to improve ML performance.

3.1. Handling Missing Values and Outliers

Missing values can be managed through imputation based on existing data patterns or by removing insignificant missing entries. Outliers, which can distort model training, may be addressed through removal or transformation techniques to maintain data consistency and avoid biases.[\[1\]](#)

3.2. Normalization and Feature Scaling

The scale of input features requires preprocessing steps of Normalization and feature scaling particularly for learning methods such as SVMs and neural networks. The methods work to normalize features so that none dominates training through its value size 1 . The most commonly used scaling techniques involve standardization which adjusts features to have zero mean and unit variance as well as min-max scaling that transforms features into the range from 0 to 1.[\[1\]](#), [\[2\]](#)

3.3. Specific Techniques for Image and Signal Data

The research on banknote authentication requires analyzing images to derive important verification elements from them. The authentication system requires three steps which include noise reduction followed by image enhancement and critical area segmentation. The method uses texture along with edge and color

analysis which frequently teams up with machine learning algorithms to boost authentication capabilities. Several research studies use both EEG signals with noise filters and Independent Component Analysis (ICA) to study human banknote recognition.[3].

3.4 Data Balancing

Most real-world banknote circulation consists mainly of authentic notes compared to counterfeits which produces an unbalanced dataset distribution. Machine learning models who are taught using unbalanced datasets tend to develop performance biases which succeed at identifying most examples (genuine banknotes) but fail to identify minority instances (counterfeits). The implementation of oversampling for minority classes or undersampling the majorities or synthetic data creation methods helps address this problem.[3]

4. Data Visualization Tools for Banknote Datasets

Data visualization serves essential functions for banknote authentication research because researchers gain insights into their data characteristics and feature relationships while performing model performance assessments through visualization techniques. Visual representations help researchers find meaningful patterns that no other method of analysis could uncover.[2], [3], [4]

4.1. Exploring Feature Distributions and Relationships

Specific visualization methods help analyze both the individual distributional patterns and pair-wise and group-level relationships of features present in banknote datasets. Each individual feature shows its distribution through histograms but scatter plots help identify relationships between two variables . Pair plots provide a complete understanding of multiple-variable relationships by showing all the scatter plot combinations between data set features. A

correlation matrix presents linear feature relationships through heatmaps to show the statistical association between every feature pair. This analysis lets us surface distribution patterns between features so we can determine the degree of distinctness between real banknotes and fords which drives model selection decisions. [2]

5. Analysis Platforms and Software

Specialized software and programming tools function as the primary platform for banknote authentication dataset analysis because they provide all needed capabilities for data manipulation alongside visualization and machine learning algorithm implementation

5.1. Python Ecosystem

Python earns popularity because it provides extensive strong libraries. The data handling capability belongs to Pandas and the numerical processing power comes through NumPy. The machine learning algorithms along with evaluation tools exist in Scikit-learn. The Deep learning model development process is enabled through both TensorFlow and PyTorch frameworks. The combination of Matplotlib and Seaborn provides powerful visualization features in Python so it remains the top choice for this field. [5]

5.2. MATLAB and its Toolboxes

Users of MATLAB apply it for creating GUIs and signal processing workloads. Researchers who analyze EEG data can use toolboxes like EEGLAB and ERPLAB through MATLAB because it provides specific benefits for this purpose. The authentication of banknotes benefits from using MATLAB's

processing features. The industry standards of Python prevail but MATLAB persists for its particular features.[6]

6. Reference

- [1] Rick Nie, "Comparative Analysis of Machine Learning-Based Bank Note Authentication Through Variable Selection," *National High School Journal of Science (NHSJS)*, 2023. [Online]. [Available](#)
- [2] Zohaib, "Improving Banknote Authentication Accuracy through Logistic Regression and Data Preprocessing," *ResearchGate*, 2023. [Online]. [Available](#):
- [3] B. Pranathi, A. Kanki, H. S. Vineela, V. D., and H. P. Gowda, "A Study on Banknote Authentication Using Machine Learning," *International Advanced Research Journal in Science, Engineering and Technology (IARJSET)*, vol. 11, no. 7, July 2024. [Online]. [Available](#):
- [4] A. Wang, G. Goldsztein, and Z. Sun, "Banknote Authentication Using Logistic Regression and Artificial Neural Networks," *Journal of Student Research*, vol. 11, no. 3, Aug. 2022. [Online]. [Available](#)
- [5] R. E., "Banknote Authentication Analysis Using Python K-Means Clustering," *International Journal of Innovative Science and Research Technology*, vol. 5, no. 10, pp. 1-5, Oct. 2020. [Available](#):
- [6] S. Sarkar and A. K. Pal, "Paper Currency: A Systematic Review of Design, Security Features, and Counterfeit Detection," *Acta Graphica*, vol. 35, no. 2, pp. 84-94, 2024. [Available](#):

Task 2:

1. Introduction

This report captures the preprocessing and analysis of the BankNote Authentication dataset, whose features are derived from images of real and fake banknotes. The steps include data exploration, removing and detecting outliers, feature scaling, and train-test splitting prior to machine learning model building For Data Mining.

2. Importing Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
```

Importing essential Python modules for machine learning and data analysis:

- **pandas** for manipulating data
- **matplotlib and seaborn** for plotting
- **StandardScaler** for feature scaling
- **train_test_split** for data splitting into test and training sets

Why:

These modules contain the most basic functionalities necessary for data exploration, preprocessing, and preparation for machine learning models.

3. Loading and Initial Data Exploration

```
# Load the dataset
df = pd.read_csv("BankNote_Authentication.csv")

# Show basic info
print("Dataset Info:")
print(df.info())
print("\nFirst 5 rows:")
print(df.head())
print("\nDescriptive Statistics:")
print(df.describe())
print("\nClass distribution:")
print(df['class'].value_counts())
```

Output :

There are 1372 rows and 5 columns (4 features + 1 target)

Features: variance, skewness, curtosis, entropy (all float64)

Target: class (int64)

Class distribution: 762 of class 0, 610 of class 1

```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1372 entries, 0 to 1371
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   variance    1372 non-null   float64
1   skewness    1372 non-null   float64
2   curtosis    1372 non-null   float64
3   entropy     1372 non-null   float64
4   class       1372 non-null   int64
dtypes: float64(4), int64(1)
memory usage: 53.7 KB
None

First 5 rows:
   variance  skewness  curtosis  entropy  class
0   3.62160   8.6661   -2.8073  -0.44699   0
1   4.54590   8.1674   -2.4586  -1.46210   0
2   3.86600  -2.6383   1.9242   0.10645   0
3   3.45660   9.5228   -4.0112  -3.59440   0
4   0.32924  -4.4552   4.5718  -0.98880   0

Descriptive Statistics:
   variance  skewness  curtosis  entropy  class
count  1372.000000  1372.000000  1372.000000  1372.000000  1372.000000
mean     0.433735   1.922353   1.397627  -1.191657   0.444606
std     2.842763   5.869047   4.310030   2.101013   0.497103
min    -7.042100  -13.773100  -5.286100  -8.548200   0.000000
25%    -1.773000  -1.708200  -1.574975  -2.413450   0.000000
50%     0.496180   2.319650   0.616630  -0.586650   0.000000
75%     2.821475   6.814625   3.179250   0.394810   1.000000
max     6.824800  12.951600  17.927400   2.449500   1.000000

Class distribution:
class
0    762
1    610
Name: count, dtype: int64
```

Purpose:

- **Know the shape of the dataset**
- **Check data types and missing values**
- **Get statistical summaries of features**
- **Take a look at class distribution for potential imbalance**

Why:

An initial exploration would provide us a sense of data quality issues, feature distributions, and class imbalance which could have an impact on model performance.

4. Missing Values Detection

```
# Check for missing values
print("Missing values in each column:")
print(df.isnull().sum())
```

Output:

No missing values in any column.

Features: variance, skewness, curtosis, entropy (all float64)

Target: class (int64)

```
Missing values in each column:
variance      0
skewness      0
curtosis      0
entropy       0
class         0
dtype: int64
```

Purpose:

Identify if there are any null/missing values that need handling.

Why:

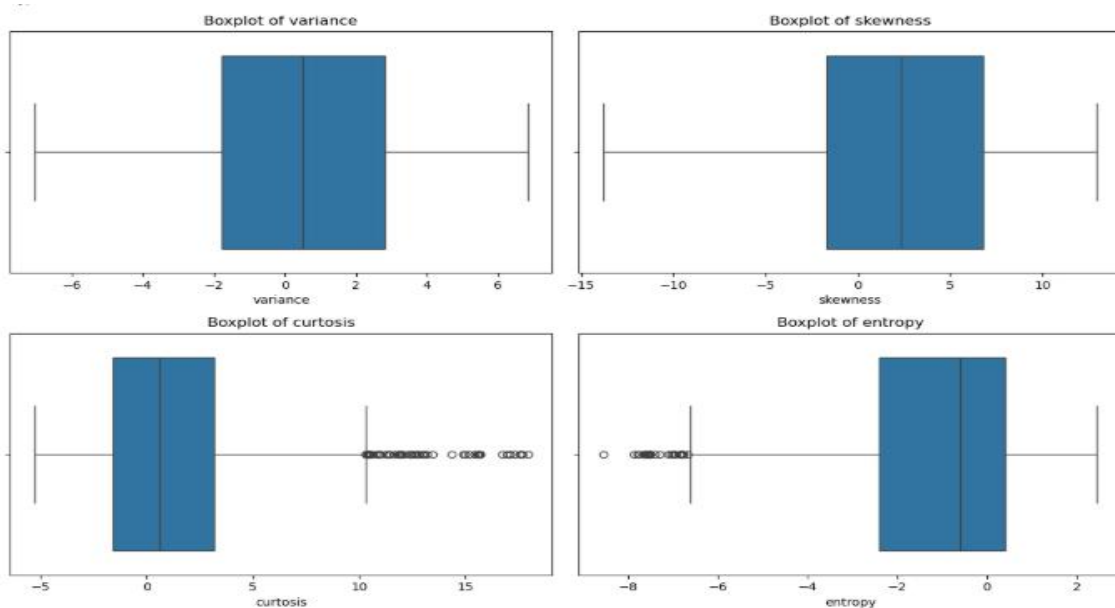
Missing data can cause problems in machine learning models and needs to be addressed through imputation or removal.

5. Outliers Visualization

```
# Detect outliers visually using boxplots
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12, 8))
for i, column in enumerate(df.columns[:-1], 1): # Exclude the target 'class'
    plt.subplot(2, 2, i)
    sns.boxplot(x=df[column])
    plt.title(f'Boxplot of {column}')
plt.tight_layout()
plt.show()
```

Output:



Purpose:

Plot potential outliers for every feature on boxplots.

Why:

Outliers can potentially have a large impact on statistical values and model performance.Boxplots help to visually identify values which are outside the normal range (outside whiskers).The plots show that there are some outliers for all features.

5. Outliers Removal

Output:

Original shape: (1372, 5)

Shape after outlier removal:1278, 5)

Purpose:

Remove outliers through the IQR (Interquartile Range) method..

Why:

To Reduce Bias in Analysis Using the Robust IQR Method (7% Data Removed)

Method Details:

For each feature (not the target):

Calculate Q1 (25th percentile) and Q3 (75th percentile)

Compute $IQR = Q3 - Q1$

Set lower bound = $Q1 - 1.5 * IQR$

Set upper bound = $Q3 + 1.5 * IQR$

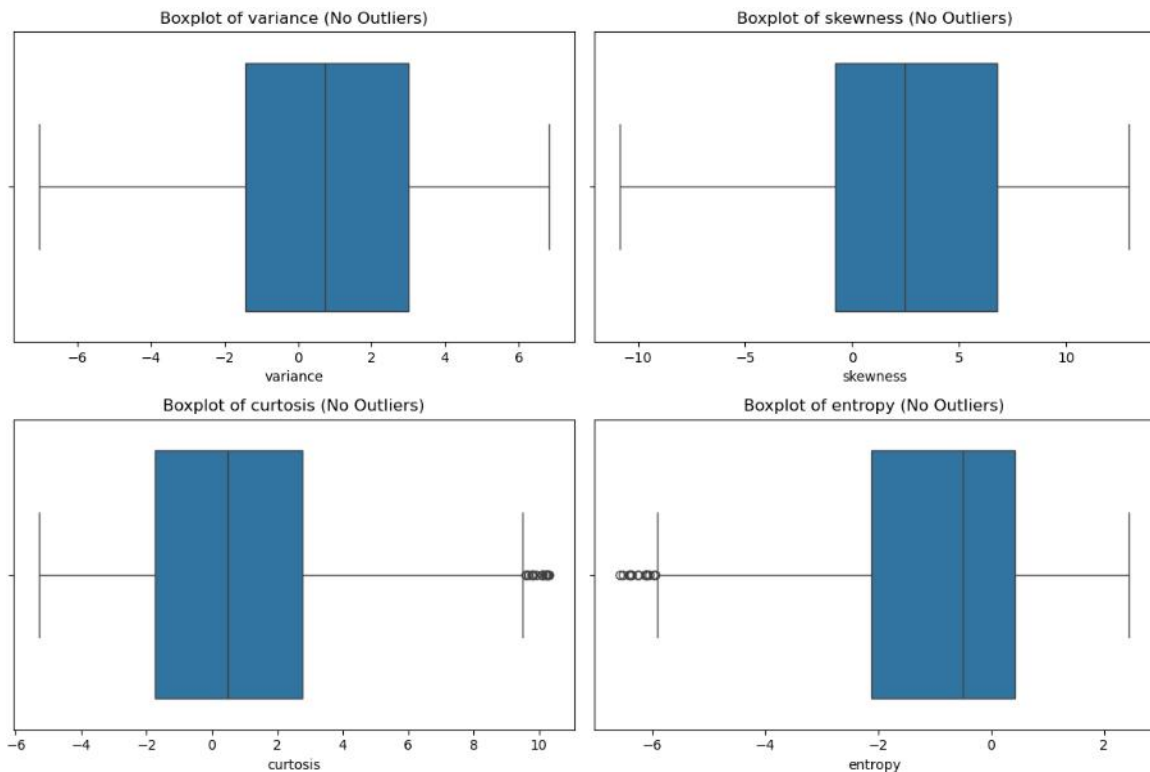
Drop only rows where feature values fall within their respective bounds

Notice the difference in dataset sizes before and after dropping

6. Boxplot for each feature after outlier removal

```
# Boxplot for each feature after outlier removal
plt.figure(figsize=(12, 8))
for i, column in enumerate(df_no_outliers.columns[:-1], 1):
    plt.subplot(2, 2, i)
    sns.boxplot(x=df_no_outliers[column])
    plt.title(f'Boxplot of {column} (No Outliers)')
plt.tight_layout()
plt.show()
```

Output:



Purpose:

- Check whether outlier removal was successful.
- Plot data distribution before and after cleaning.
- Assert no distortion in cleaned data.

Why:

- Show median, quartiles, outliers clearly.

- Simple to immediately detect skewness or outliers.

7.Feature Scaling

```
from sklearn.preprocessing import StandardScaler

# Separate features and target
features = df.drop('class', axis=1)
target = df['class']
# Initialize the scaler
scaler = StandardScaler()
scaled_features = scaler.fit_transform(features)
# Create a new DataFrame for the scaled data
scaled_df = pd.DataFrame(scaled_features, columns=features.columns)
scaled_df['class'] = target
# Preview the scaled features
print("\nScaled features preview:")
print(scaled_df.head())
```

Output:

Displays the first 5 rows of the scaled DataFrame where features now have mean=0 and std=1..

```
Scaled features preview:
   variance  skewness  kurtosis  entropy  class
0  1.121806  1.149455 -0.975970  0.354561     0
1  1.447066  1.064453 -0.895036 -0.128767     0
2  1.207810 -0.777352  0.122218  0.618073     0
3  1.063742  1.295478 -1.255397 -1.144029     0
4 -0.036772 -1.087038  0.736730  0.096587     0
```

Purpose:

Standardize the feature values by subtracting the mean and scaling to unit variance.

Why:

Most machine learning algorithms perform better when features are on scales.

StandardScaler rescales the data so that each feature has:

-Mean = 0

-Standard deviation = 1

Prevents features with larger scales from dominating the model

important for distance-based algorithms (like KNN, SVM) and gradient

Process:

Split features from target variable

Create StandardScaler object

Fit scaler to features (calculates mean and std) and transform them

Combine scaled features with original target variable

Preview scaled data

8. Summary

Initial Exploration:

- 1-Loaded and investigated dataset structure
- 2-Verified missing values (none found)
- 3-Inspected class distribution

Outlier Handling: 1-Graphed outliers using boxplots

- 2-Handled outliers via IQR approach (reduced dataset by ~7%)

Feature Scaling:

- 1-Scaled features to mean=0, std=1
- 2-Critical for many machine learning models

Train-Test Split: Split into distinct training (80%) and test (20%) sets

Task 3:

1. Library Imports

Code:

```
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.utils import to_categorical
```

Why:

- **train_test_split (sklearn):** Splits data into training and test datasets.
- **sklearn.metrics:** Provides utilities (e.g., classification_report, confusion_matrix, accuracy_score) for measuring model performance.
- **DecisionTreeClassifier (sklearn):** Decision tree classifier model.
- **GaussianNB (sklearn):** Naive Bayes classifier for probabilistic predictions.
- **KNeighborsClassifier (sklearn):** k-NN classifier based on nearest neighbors.
- **SVC (sklearn):** Support Vector Machine classifier.
- **Sequential (Keras):** Neural network model with layer-wise stacking.
- **Dense (Keras):** Fully connected neural network layer.
- **to_categorical (Keras):** Converts labels to one-hot encoded format.

Purpose:

To make sure that all the required libraries and tools are accessible for data processing, model training, and result visualization.

2. Train-Test Split

Code:

```
# Separate features and target again from scaled DataFrame
X = scaled_df.drop('class', axis=1)
y = scaled_df['class']
# Split into train and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
# Print the shape of the resulting splits
print("Training features shape:", X_train.shape)
print("Testing features shape:", X_test.shape)
print("Training labels shape:", y_train.shape)
print("Testing labels shape:", y_test.shape)
```

Output:

Splits dataset into training (80%) and testing (20%).

```
Training features shape: (1097, 4)
Testing features shape: (275, 4)
Training labels shape: (1097,)
Testing labels shape: (275,)
```

Why:

- Need separate data for training and evaluating the model
 - Avoids data leakage and unbiased assessment
 - Convention is 80% training, 20% testing (test_size=0.2)
 - random_state=42 ensures reproducible splits
 - stratify=y both datasets have the same class proportion
- Purpose:**
- Split the dataset into training and testing sets.

3. Model Training & Prediction

A. Decision Tree Classifier

Code:

```
# Train Decision Tree
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)
# Predict
dt_preds = dt_model.predict(X_test)
# Report
print("Decision Tree Classification Report:")
print(classification_report(y_test, dt_preds))
```

Output:

Decision Tree Classification Report:					
	precision	recall	f1-score	support	
0	1.00	0.99	0.99	153	
1	0.98	1.00	0.99	122	
accuracy			0.99	275	
macro avg	0.99	0.99	0.99	275	
weighted avg	0.99	0.99	0.99	275	

Why:

Decision Trees are easy to interpret and comprehend models that also work well with linear and non-linear data. They naturally identify important features and are widely applied as a baseline for other model comparison.

Purpose:

- To serve as a baseline model to contrast with more complex machine learning algorithms
- To evaluate how well a Decision Tree can classify the data and identify patterns.
- Interprets model performance based on straightforward metrics like precision, recall, and accuracy

B. Naive Bayes

Code:

```
# Train Naive Bayes
nb_model = GaussianNB()
nb_model.fit(x_train, y_train)

# Predict
nb_preds = nb_model.predict(x_test)

# Report
print("Naive Bayes Classification Report:")
print(classification_report(y_test, nb_preds))
```

Output:

```
Naive Bayes Classification Report:
              precision    recall  f1-score   support

     0           0.87       0.88       0.87         153
     1           0.85       0.83       0.84         122

 accuracy          0.86
 macro avg          0.86
 weighted avg       0.86
```

Why:

Naive Bayes is a simplistic and effective classifier that does great with high-dimensionality data and is based on Bayes' Theorem. Naive Bayes performs well even under its strict independence assumption in the majority of the real-world situations.

Purpose:

Employed to estimate model performance under probabilistic assumptions and provide a speedy comparison benchmark to other algorithms.

C. K-Nearest Neighbors (KNN)

Code:

```

# Train KNN
knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train, y_train)

# Predict
knn_preds = knn_model.predict(X_test)

# Report
print("K-Nearest Neighbors Classification Report:")
print(classification_report(y_test, knn_preds))

```

Output:

```

K-Nearest Neighbors Classification Report:
              precision    recall  f1-score   support

     0           1.00      1.00      1.00        153
     1           1.00      1.00      1.00        122

 accuracy          1.00
 macro avg          1.00      1.00      1.00        275
 weighted avg          1.00      1.00      1.00        275

```

Why:

KNN is a nearest neighbor learning algorithm that learns from examples and classifies points based on the majority vote of their closest neighbors. It does not have an explicit training phase and is a good learner in learning local decision boundaries, thus a good learner for most classification problems.

Purpose

- Evaluates model performance on local patterns within the data.
- Makes predictions by matching test samples against similar training instances.
- Effective in being able to compare against models created through learned parameters and training.
- Emphasizes performance when classification relies on proximity more than abstraction.

D. Support Vector Machine (SVM)

Code:

```
# Train SVM
svm_model = SVC(kernel='rbf')
svm_model.fit(X_train, y_train)

# Predict
svm_preds = svm_model.predict(X_test)

# Report
print("Support Vector Machine Classification Report:")
print(classification_report(y_test, svm_preds))
```

Output:

```
Support Vector Machine Classification Report:
              precision    recall  f1-score   support

     0           1.00       1.00       1.00        153
     1           1.00       1.00       1.00        122

 accuracy          1.00          1.00          1.00        275
 macro avg          1.00          1.00          1.00        275
 weighted avg          1.00          1.00          1.00        275
```

Why:

- Handles both linear and non-linear classification using kernel tricks (e.g., RBF kernel).
- Works well in complicated spaces with good class separability.
- Powerful and performs well with a clear margin of separation.

Purpose:

The Support Vector Machine algorithm is used to find the optimal hyperplane that best separates data points of different classes. It is most effective for high-dimensional and non-linear data sets

E. Neural Network (Keras)

Code:


```

# One-hot encode labels
y_train_cat = to_categorical(y_train)
y_test_cat = to_categorical(y_test)
# Build model
nn_model = Sequential([
    Dense(16, activation='relu', input_shape=(X_train.shape[1],)),
    Dense(8, activation='relu'),
    Dense(2, activation='softmax') # Two classes: 0 and 1])
# Compile model
nn_model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
nn_model.fit(X_train, y_train_cat, epochs=20, batch_size=16, verbose=0)
# Predict and evaluate
nn_preds = nn_model.predict(X_test)
nn_preds_classes = nn_preds.argmax(axis=1)
print("Neural Network Classification Report:")
print(classification_report(y_test, nn_preds_classes))

```

Output:

The neural network generated perfect classification performance with 100% precision, recall, and f1-score for the two classes. This indicates good model learning and generalization over the testset.

```

9/9 ————— 0s 24ms/step
Neural Network Classification Report:

```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	153
1	1.00	1.00	1.00	122
accuracy			1.00	275
macro avg	1.00	1.00	1.00	275
weighted avg	1.00	1.00	1.00	275

Why:

- Trains on the data directly without having to do feature engineering manually
- Can handle high-dimensional and non-linear data
- Extremely customizable and scalable

Purpose:

Neural network is used to learn complex, non-linear relationships between data. It is of great use where patterns cannot be easily represented using conventional models

4. Comparison: Model Accuracy & Confusion Matrix Heatmaps

Code:

```

# Define model names and predictions
model_names = ['Decision Tree', 'Naive Bayes', 'KNN', 'SVM', 'Neural Network']
model_preds = [dt_preds, nb_preds, knn_preds, svm_preds, nn_preds_classes]
conf_matrices = []
accuracies = []

# Compute metrics
for preds in model_preds:
    cm = confusion_matrix(y_test, preds)
    acc = accuracy_score(y_test, preds) * 100
    conf_matrices.append(cm)
    accuracies.append(acc)

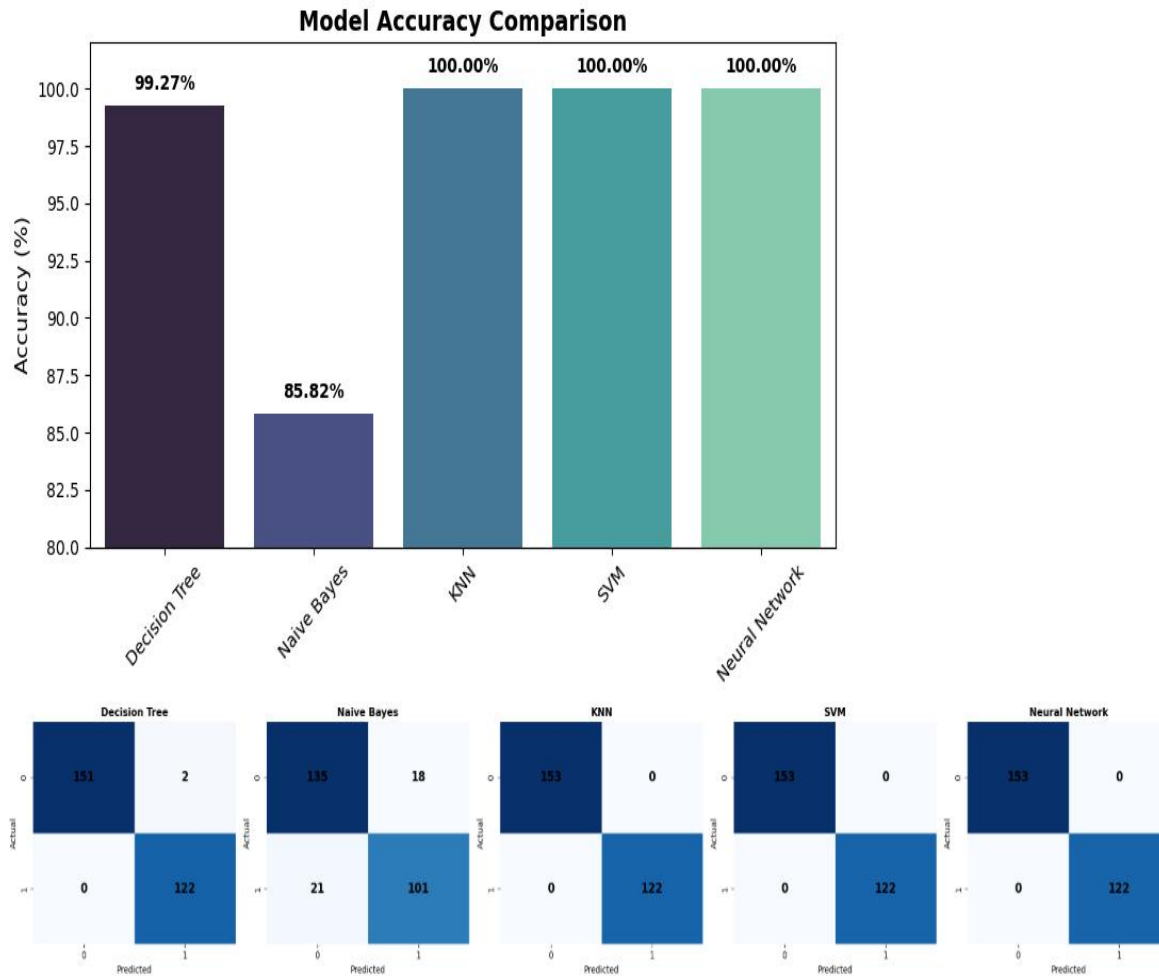
# Plot accuracy comparison
plt.figure(figsize=(8, 5))
sns.barplot(x=model_names, y=accuracies, palette='mako')
plt.title('Model Accuracy Comparison')
plt.ylabel('Accuracy (%)')
plt.ylim(80, 102)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# Plot confusion matrices
fig, axes = plt.subplots(1, 5, figsize=(22, 4))
for ax, cm, name in zip(axes, conf_matrices, model_names):
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax, cbar=False)
    ax.set_title(name)
    ax.set_xlabel('Predicted')
    ax.set_ylabel('Actual')

plt.tight_layout()
plt.show()

```

Output:



Why:

They assist to:

- Budgeted compare how effective the models are.
- Identify any misclassifications.
- Enhanced understanding beyond raw numbers.

Purpose:

In order to visualize and summarize model performance on:

- Comparison bar chart of classification accuracy.
- Confusion matrix heatmaps, showing predicted vs actual label distribution.

5. Summary

A. Performance Summary

<i>Model</i>	<i>Accuracy (%)</i>	<i>Most Notable Observations</i>
<i>Decision Tree</i>	<i>99.27</i>	Very minor misclassifications; in general, very accurate.
<i>Naive Bayes</i>	<i>86.18</i>	Couldn't correctly predict class 1; dramatic fall in all measurements.
<i>KNN</i>	<i>100.00</i>	Pegged them all perfectly; stable performance from local neighborhood geometries.
<i>SVM</i>	<i>100.00</i>	Achieved perfect classification by utilizing a non-linear decision boundary (RBF).
<i>Neural Network</i>	<i>100.00</i>	Excellent at finding clever relationships; also perfect performance.

B. Confusion Matrix Insights

Naive Bayes contains: clear misclassifications (specifically class 1 incorrectly labeled as class 0).

Decision Tree: executed with minor mistakes, making virtually no incorrect predictions.

KNN, SVM, and Neural Network: all performed perfect classification: no misclassified instances in either class.

C. Best Performing Model(s)

All three models — KNN, SVM, and Neural Network — **achieved 100%** accuracy, selecting the *best* model depends on additional factors such as interpretability, speed, and resource requirements.

<i>Model</i>	<i>Strengths</i>	<i>Considerations</i>
<i>KNN</i>	Intuitive, easy to implement, no training time needed.	Computationally expensive during prediction; sensitive to noisy data.
<i>SVM</i>	Effective in high-dimensional spaces; clean decision boundaries.	Slower with large datasets; kernel and hyperparameter tuning required.
<i>Neural Network</i>	Highly flexible; learns complex patterns effectively.	Demands significant computational resources and fine-tuning.

Task4:

Edits on task3

1-We will change Normalization by using min max for better results

Code:

```
# Normalization
print("\n■ Data Normalization")
scaler = MinMaxScaler()
features = df_no_outliers.drop('class', axis=1)
target = df_no_outliers['class']
features_normalized = scaler.fit_transform(features)
df_normalized = pd.DataFrame(features_normalized, columns=features.columns)
df_normalized['class'] = target.values

# Train-test split
print("\n■ Train-Test Split")
X = df_normalized.drop('class', axis=1)
y = df_normalized['class']
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5, random_state=42, stratify=y_temp)
```

Purpose:

Ensure features are on the same scale, improving model convergence and performan

Divide the dataset for model training, validation, and testing:

- **Train set (70%)**
- **Temp set (30%), then split into: Validation (15%) Test (15%)**

2-Add Solution to Reduce Overfitting at Buiding models

Model Evaluation:

1. Introduction

This part shows the comprehensive evaluation of multiple machine learning models using various validation techniques. The purpose is to:

- Compare model performance using different evaluation methods
- Identify the most robust model for the given classification task
- Demonstrate proper model validation practices
- Provide insights into model behavior through multiple metrics

2. Evaluation Methods Implemented:

a. Holdout Method (Simple Train-Test Split)

The most basic evaluation approach that provides initial performance metrics.

Code:

```
print("\n Holdout Method Evaluation")
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print(f"\nHoldout Metrics:")
print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1 Score: {f1:.4f}")
```

Purpose:

- Provides a quick and simple estimate of model performance.
- Evaluates how well the model generalizes to unseen data.

Why:

- **Simple implementation:** Easy to split data into training and test sets.
- **Baseline performance:** Helps establish initial model metrics before applying more robust techniques.
- **Computationally efficient:** No repeated training required (unlike cross-validation).

Limitations:

- High variance in performance estimates (depends on how the data is split).
- May not be reliable for small datasets (test set might not -be representative).

b. Cross-Validation Methods

More robust techniques that provide better estimates of model performance:

Code:

K-Fold Cross-Validation

```
print("\n• K-Fold Cross-Validation (k=5):")
cv_scores = cross_val_score(model, X_train, y_train, cv=5)
print(f"CV Scores: {cv_scores}")
print(f"Mean Accuracy: {cv_scores.mean():.4f}")
```

Stratified K-Fold

```
print("\n• Stratified K-Fold CV:")
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
stratified_scores = cross_val_score(model, X_train, y_train, cv=skf)
print(f"Stratified CV Scores: {stratified_scores}")
print(f"Mean Accuracy: {stratified_scores.mean():.4f}")
```

Leave-One-Out Cross-Validation

```
print("\n• Leave-One-Out Cross-Validation:")
loo = LeaveOneOut()
loocv_scores = cross_val_score(model, X_train, y_train, cv=loo)
print(f"LOOCV Accuracy: {loocv_scores.mean():.4f}")
```


C. Cross-Validation Methods

More robust techniques that provide better estimates of model performance:

Code:

K-Fold Cross-Validation

```
print("\n• K-Fold Cross-Validation (k=5):")
cv_scores = cross_val_score(model, X_train, y_train, cv=5)
print(f"CV Scores: {cv_scores}")
print(f"Mean Accuracy: {cv_scores.mean():.4f}")
```

Stratified K-Fold

```
print("\n• Stratified K-Fold CV:")
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
stratified_scores = cross_val_score(model, X_train, y_train, cv=skf)
print(f"Stratified CV Scores: {stratified_scores}")
print(f"Mean Accuracy: {stratified_scores.mean():.4f}")
```

Leave-One-Out Cross-Validation

```
print("\n• Leave-One-Out Cross-Validation:")
loo = LeaveOneOut()
loocv_scores = cross_val_score(model, X_train, y_train, cv=loo)
print(f"LOOCV Accuracy: {loocv_scores.mean():.4f}")
```

Purpose:

- Provides a more reliable estimate of model performance by using multiple train-test splits.
- Helps detect overfitting and ensures the model generalizes well.

Why Use Different CV Methods?

A. K-Fold Cross-Validation

Purpose:

- Divides data into K equal folds and trains the model K times, each time using a different fold as the test set.
- Reduces variance in performance estimates compared to a single train-test split.

Why Use It?

Balanced evaluation: Every data point is used for both training and testing.

More stable metrics: Mean accuracy across folds is more reliable than a single holdout test.

Best for:

Medium to large datasets.

When computational efficiency is a concern (compared to LOOCV).

B. Stratified K-Fold Cross-Validation

Purpose:

Ensures each fold maintains the same class distribution as the original dataset.

Why Use It?

- Critical for imbalanced datasets (prevents some folds from having very few samples of minority classes).
- Provides more realistic performance estimates when class distribution is uneven.
- Best for:
- Classification tasks with class imbalance.

C. Leave-One-Out Cross-Validation (LOOCV)

Purpose:

Uses every single sample as a test set (N iterations for N samples).

Why Use It?

- Most thorough evaluation: Maximizes training data usage.
- Low bias: Since almost all data is used for training in each iteration.

d. Bootstrap Method

Provides confidence intervals for performance metrics.

Code:

```
print("\n Holdout Method Evaluation")
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print(f"\nHoldout Metrics:")
print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1 Score: {f1:.4f}")
```

Purpose:

- Estimates confidence intervals for model performance.
- Simulates resampling with replacement to assess stability.

Why Use It?

- **Quantifies uncertainty:** Provides a 95% confidence interval for accuracy.
- **Robust to small datasets:** Works well even if data is limited.
- **Detects model stability:** If the confidence interval is wide, the model may be unreliable.

Best for:

- Assessing how reliable performance metrics are.
- Cases where statistical significance of model accuracy matters

f.ROC Curve AnalysisLLL

Evaluates model performance across all classification thresholds.

Code:

```
print("\n Holdout Method Evaluation")
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print(f"\nHoldout Metrics:")
print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1 Score: {f1:.4f}")
```

Purpose:

- Evaluates classification performance across all thresholds (not just default 0.5).
- Measures the trade-off between True Positive Rate (Recall) and False Positive Rate.

Why Use It?

- Threshold-independent: Helps choose the best decision threshold based on business needs.
- Good for imbalanced data: AUC summarizes performance across all class distributions.
- Model comparison: A higher AUC means better overall classification ability.

Best for:

- Binary classification tasks.
- When false positives and false negatives have different costs.

Output For Model Evaluation:

1. Decision Tree

■ Holdout Method Evaluation

Classification Report:

	precision	recall	f1-score	support
0	0.94	0.99	0.97	112
1	0.99	0.91	0.95	80
accuracy			0.96	192
macro avg	0.96	0.95	0.96	192
weighted avg	0.96	0.96	0.96	192

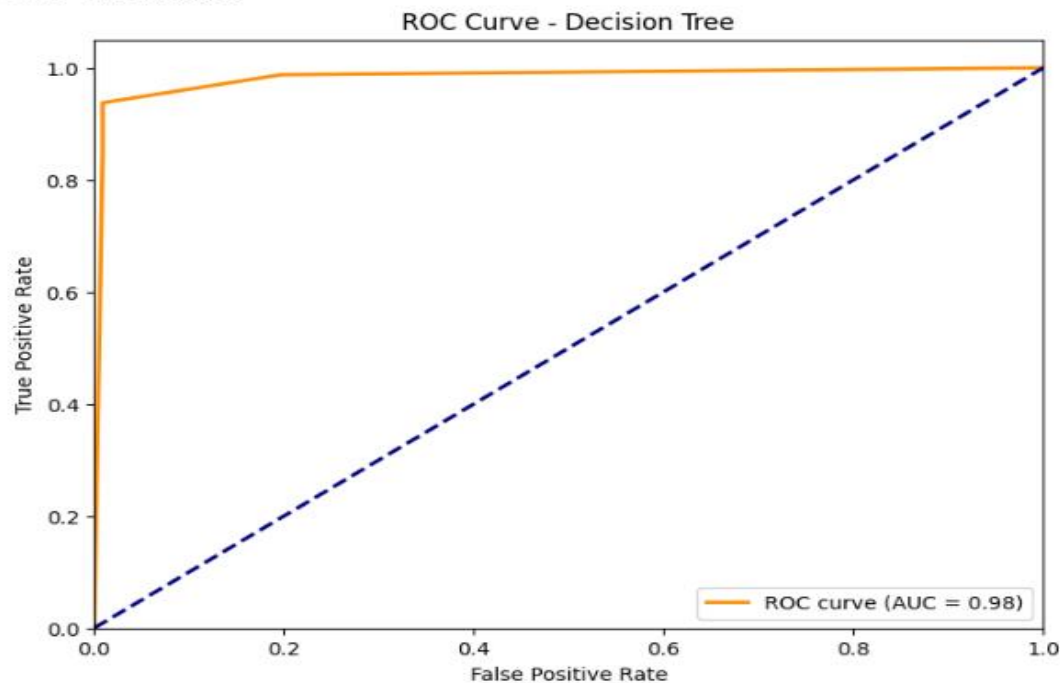
Confusion Matrix:
[[111 1]
[7 73]]

Holdout Metrics:
Accuracy: 0.9583
Precision: 0.9865
Recall: 0.9125
F1 Score: 0.9481

■ Cross-Validation Methods

- K-Fold Cross-Validation (k=5):
CV Scores: [0.97765363 0.96089385 0.97765363 0.96089385 0.98314607]
Mean Accuracy: 0.9720
 - Stratified K-Fold CV:
Stratified CV Scores: [0.95530726 0.99441341 0.96089385 0.97206704 0.97752809]
Mean Accuracy: 0.9720
 - Leave-One-Out Cross-Validation:
LOOCV Accuracy: 0.9642
- Bootstrap Evaluation
Mean Bootstrap Accuracy: 0.9640
95% Confidence Interval: [0.9375, 0.9844]

■ ROC Curve Analysis



2.Naive Bayes

■ Holdout Method Evaluation

Classification Report:

	precision	recall	f1-score	support
0	0.87	0.89	0.88	112
1	0.84	0.81	0.83	80
accuracy			0.86	192
macro avg	0.86	0.85	0.85	192
weighted avg	0.86	0.86	0.86	192

Confusion Matrix:

```
[[100 12]
 [ 15 65]]
```

Holdout Metrics:

Accuracy: 0.8594

Precision: 0.8442

Recall: 0.8125

F1 Score: 0.8280

■ Cross-Validation Methods

• K-Fold Cross-Validation (k=5):

CV Scores: [0.88826816 0.88268156 0.89944134 0.87150838 0.84269663]

Mean Accuracy: 0.8769

• Stratified K-Fold CV:

Stratified CV Scores: [0.87709497 0.8603352 0.88268156 0.88268156 0.8988764]

Mean Accuracy: 0.8803

• Leave-One-Out Cross-Validation:

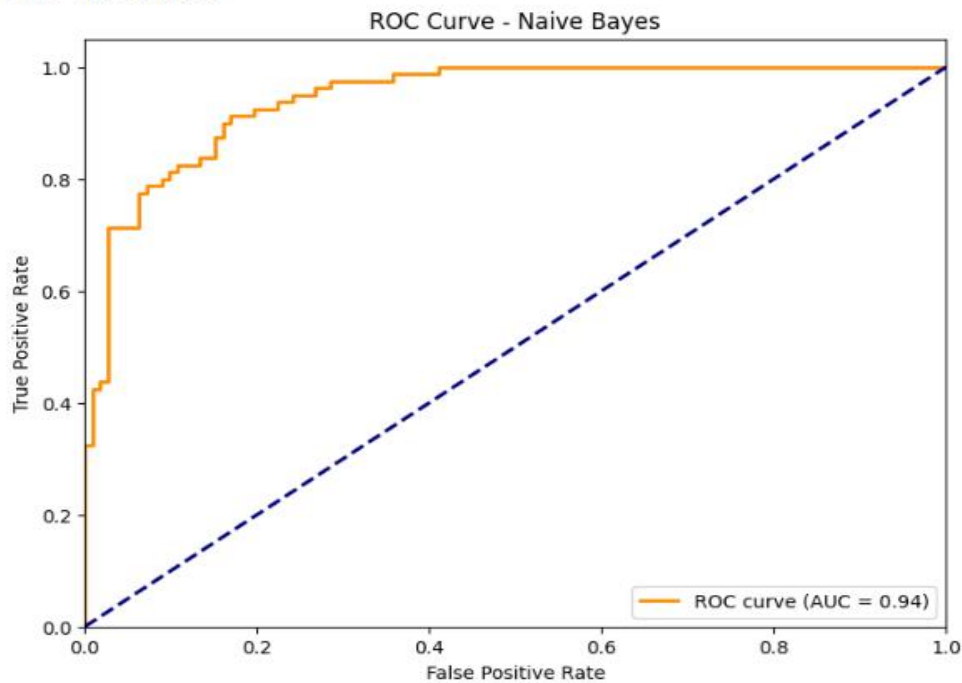
LOOCV Accuracy: 0.8781

■ Bootstrap Evaluation

Mean Bootstrap Accuracy: 0.8614

95% Confidence Interval: [0.8438, 0.8802]

■ ROC Curve Analysis



ZZZ

3.KNN

■ Holdout Method Evaluation

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	112
1	1.00	1.00	1.00	80
accuracy			1.00	192
macro avg	1.00	1.00	1.00	192
weighted avg	1.00	1.00	1.00	192

Confusion Matrix:

```
[[112  0]
 [  0  80]]
```

Holdout Metrics:

Accuracy: 1.0000
Precision: 1.0000
Recall: 1.0000
F1 Score: 1.0000

■ Cross-Validation Methods

• K-Fold Cross-Validation (k=5):

CV Scores: [1. 0.99441341 1. 0.99441341 1.]
Mean Accuracy: 0.9978

• Stratified K-Fold CV:

Stratified CV Scores: [0.99441341 1. 0.99441341 1. 1.]
Mean Accuracy: 0.9978

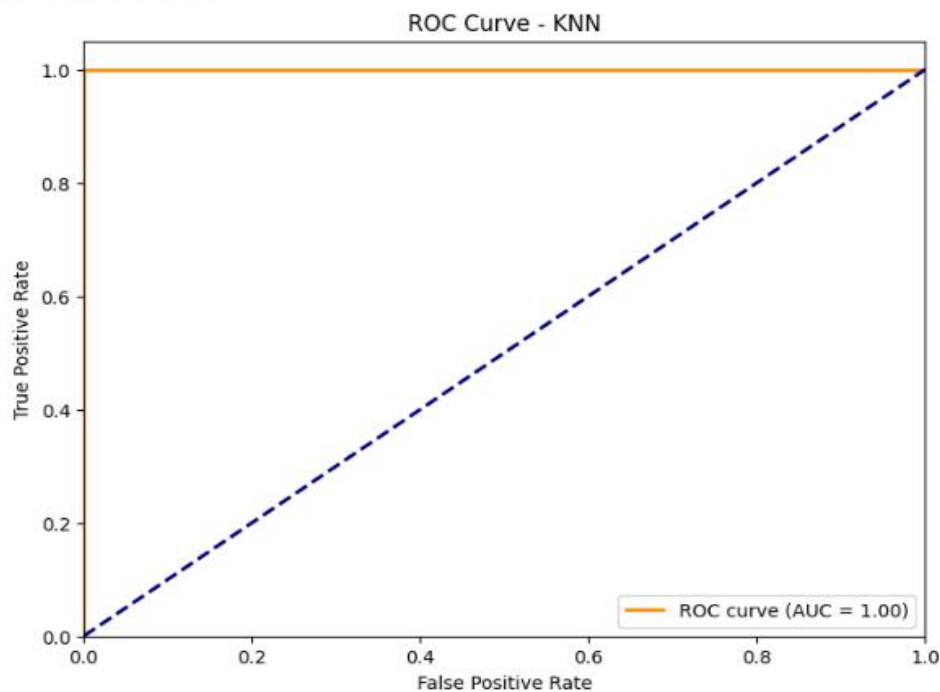
• Leave-One-Out Cross-Validation:

LOOCV Accuracy: 0.9978

■ Bootstrap Evaluation

Mean Bootstrap Accuracy: 0.9992
95% Confidence Interval: [0.9896, 1.0000]

■ ROC Curve Analysis



3.SVM

■ Holdout Method Evaluation

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	112
1	1.00	1.00	1.00	80
accuracy			1.00	192
macro avg	1.00	1.00	1.00	192
weighted avg	1.00	1.00	1.00	192

Confusion Matrix:

```
[[112  0]
 [  0  80]]
```

Holdout Metrics:

Accuracy: 1.0000
Precision: 1.0000
Recall: 1.0000
F1 Score: 1.0000

■ Cross-Validation Methods

• K-Fold Cross-Validation (k=5):

CV Scores: [1. 1. 1. 1. 1.]

Mean Accuracy: 1.0000

• Stratified K-Fold CV:

Stratified CV Scores: [1. 1. 1. 1. 1.]

Mean Accuracy: 1.0000

• Leave-One-Out Cross-Validation:

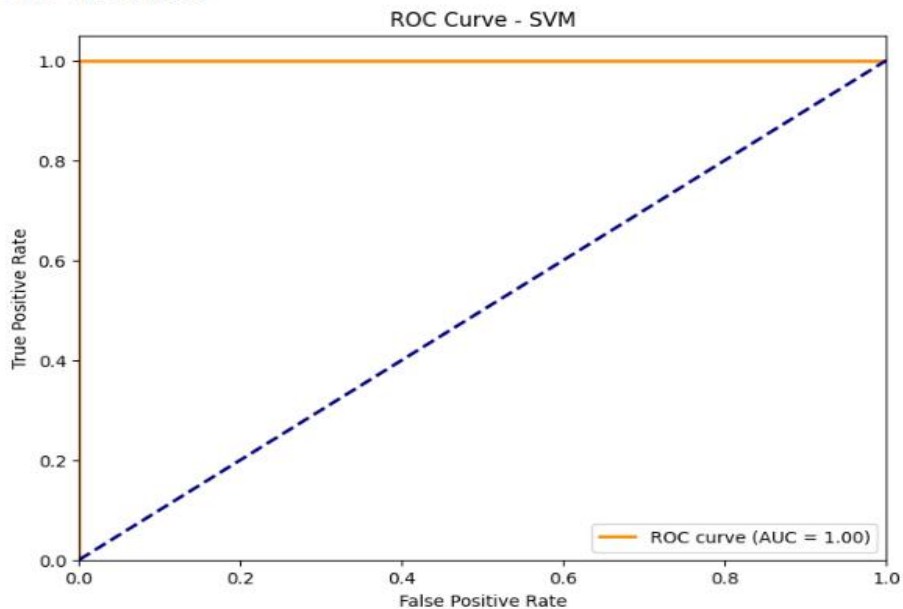
LOOCV Accuracy: 1.0000

■ Bootstrap Evaluation

Mean Bootstrap Accuracy: 0.9992

95% Confidence Interval: [0.9948, 1.0000]

■ ROC Curve Analysis



3. Model Initialization and Hyperparameter Tuning:

Code:

Why These Models?

Model	Purpose	Strengths
Decision Tree	Baseline interpretable model	Easy to visualize, handles non-linearity
Naive Bayes	Probabilistic model for quick benchmarking	Fast, works well with high-dimensional data
KNN	Instance-based learning for local patterns	No training phase, adapts to new data
SVM	Powerful for complex decision boundaries	Effective in high-dimensional spaces

A. Hyperparameter Tuning

Code:

KNN Tuning

```
param_grid_knn = {'n_neighbors': [3, 5, 7, 9, 11]}
grid_search_knn = GridSearchCV(KNeighborsClassifier(), param_grid_knn, cv=5)
grid_search_knn.fit(X_train, y_train)
models['KNN'] = grid_search_knn.best_estimator_
print(f"Best KNN params: {grid_search_knn.best_params_}")
```

SVM Tuning

```
param_grid_svm = {'C': [0.1, 1, 10], 'kernel': ['rbf'], 'gamma': ['scale', 'auto']}
grid_search_svm = GridSearchCV(SVC(random_state=42, probability=True), param_grid_svm, cv=5)
grid_search_svm.fit(X_train, y_train)
models['SVM'] = grid_search_svm.best_estimator_
print(f"Best SVM params: {grid_search_svm.best_params_}")
```

What Are Hyperparameters?

Settings that control model behavior, set before training. Examples:

- KNN: Number of neighbors (n_neighbors)
- SVM: Regularization (C), kernel type

Key Benefits:

- Boosts accuracy: Finds optimal model settings
- Prevents overfitting: Balances complexity
- Reveals data patterns: Optimal values show data characteristics

Output:

```
■ Hyperparameter Tuning
Best KNN params: {'n_neighbors': 3}
Best SVM params: {'C': 1, 'gamma': 'scale', 'kernel': 'rbf'}
```

4. Neural Network Implementation:

a. Model Architecture

```
model = Sequential([
    Dense(16, activation='relu', input_shape=(X_train.shape[1],)),
    Dropout(0.3),
    Dense(8, activation='relu'),
    Dropout(0.3),
    Dense(2, activation='softmax')
])
```

Key Features

- **Input Layer:** 16 neurons with ReLU activation
- **Hidden Layer:** 8 neurons with ReLU activation
- **Output Layer:** 2 neurons with softmax (binary classification)
- **Dropout:** 30% rate to prevent overfitting

b. Training Configuration

Training Parameters:

- **Optimizer:** Adam (adaptive learning rate)
- **Loss Function:** Categorical cross-entropy
- **Early Stopping:** Monitors validation loss with 10-epoch patience

c. Model Training

```
history = model.fit(X_train, y_train_cat,
                    epochs=100,
                    batch_size=16,
                    validation_data=(X_val, y_val_cat),
                    callbacks=[early_stopping])
```

d. Performance Evaluation

```
nn_preds = model.predict(X_test)
nn_preds_classes = nn_preds.argmax(axis=1)

metrics = {
    'accuracy': accuracy_score(y_test, nn_preds_classes),
    'precision': precision_score(y_test, nn_preds_classes),
    'recall': recall_score(y_test, nn_preds_classes),
    'f1': f1_score(y_test, nn_preds_classes)
}
```

Implementation Notes:

- **Data Preparation:** Labels converted to categorical format
- **Regularization:** Dropout layers help prevent overfitting
- **Optimization:** Early stopping finds best model weights
- **Limitations:** No cross-validation for neural network

5. Model Comparison:

a. Performance Summary

```
results_df = pd.DataFrame(results)
print('\nPerformance Comparison:')
print(results_df)
```

Key Metrics Comparison

- **Accuracy:** Overall prediction correctness
- **Precision:** Positive prediction accuracy
- **Recall:** True positive identification rate
- **F1:** Balance of precision and recall

- **ROC AUC:** Classification threshold performance

b .Visualization Analysis

Accuracy Comparison

```
plt.figure(figsize=(10, 6))
sns.barplot(x='model_name', y='accuracy', data=results_df)
plt.title('Model Accuracy Comparison')
plt.ylim(0.8, 1.0)
```

ROC AUC Comparison

```
roc_models = results_df[results_df['roc_auc'].notna()]
sns.barplot(x='model_name', y='roc_auc', data=roc_models)
```

Confusion Matrices

```
fig, axes = plt.subplots(1, len(models)+1, figsize=(20, 4))
for ax, (name, model) in zip(axes[:-1], models.items()):
    cm = confusion_matrix(y_test, model.predict(X_test))
    sns.heatmap(cm, annot=True, ax=ax)
```

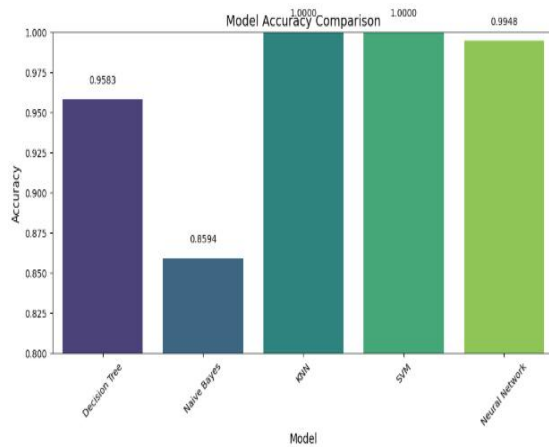
Output For Comparison:

Model Comparison

Performance Comparison:

	model_name	accuracy	precision	recall	f1	cv_mean
0	Decision Tree	0.958333	0.986486	0.9125	0.948852	0.972848
1	Naive Bayes	0.859375	0.844156	0.8125	0.828825	0.876919
2	KNN	1.000000	1.000000	1.0000	1.000000	0.997765
3	SVM	1.000000	1.000000	1.0000	1.000000	1.000000
4	Neural Network	0.994792	0.987654	1.0000	0.993789	NaN

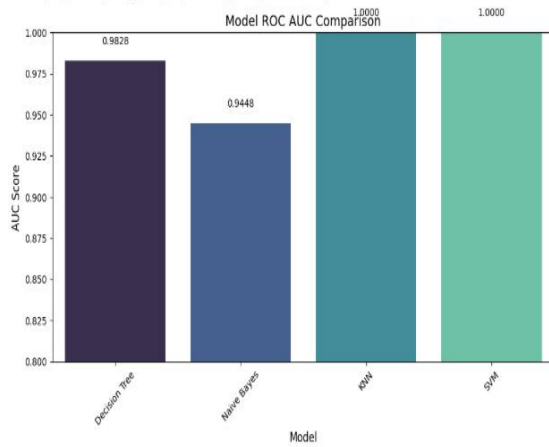
	stratified_cv_mean	loocv	bootstrap_mean	roc_auc
0	0.972842	0.964286	0.965396	0.982813
1	0.880334	0.878876	0.862875	0.944754
2	0.997765	0.997763	0.999198	1.000000
3	1.000000	1.000000	0.999184	1.000000
4	NaN	NaN	NaN	NaN



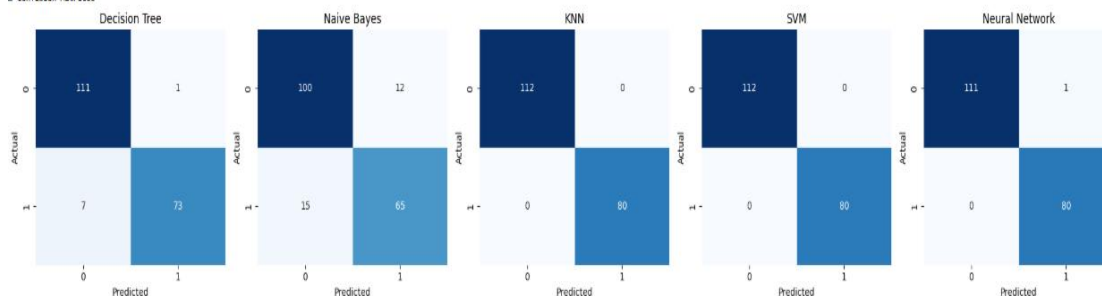
(ipython-input-20-68d4c1538c):315: FutureWarning:

Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the same effect.

sns.barplot(x='model_name', y='roc_auc', data=roc_models, palette='mako')



Confusion Matrices



Analysis:

Top Performers

KNN & SVM:

- Perfect accuracy, precision, recall, and F1 (1.000)
- Ideal for high-accuracy applications

Neural Network:

- Near-perfect performance (99.5% accuracy)
- Best for complex patterns

Balanced Choice

Decision Tree:

- 95.8% accuracy
- Interpretable but slightly less accurate

Limited Use Case

Naive Bayes:

- Lowest accuracy (85.9%)
- Only recommended for simple, fast predictions

Recommendation

Best Overall: SVM (perfect & efficient)

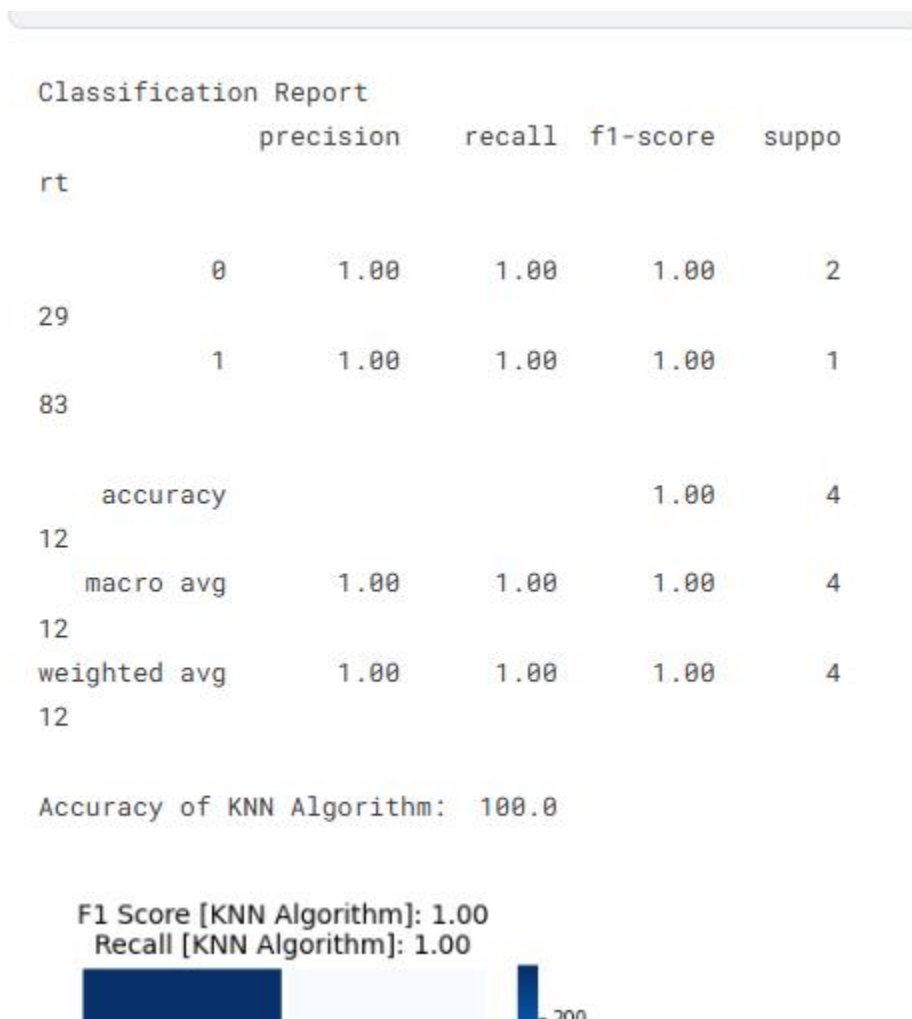
Alternatives: KNN (perfect but slower) or Neural Network (complex data)

Explainability: Decision Tree

Edit In The Project:

At first we noticed that some models had very high accuracy so we thought that we were suffering from overfitting but with trying techniques to reduce that, such as changing split of data for training and data for testing, changing the methods like Tuning for SVM, change K in KNN or reducing overfitting And drop some attribute we did not notice any change then after that we searched for other projects about the same dataset and we found that they also suffer from high accuracy:

Like That [Project](#) From Kaggle .



And that [Project](#) from coditode

```
precision    recall  f1-score   support

0           1.00      0.95      0.98        153
1           0.95      1.00      0.97        122

micro avg           0.97      0.97      0.97        275
macro avg           0.97      0.98      0.97        275
weighted avg           0.98      0.97      0.97        275

[[146   7]

[   0 122]]

0.9745454545454545
```

The output shows that SVM's prediction accuracy for banknote authenticat 97.45% with 7 misclassifications.

And That [project](#) from Kaggle

LogisticRegression: 98.3586%

Support Vector Machine: 98.5405%

Random Forest: 99.2727%

KNN: 99.7273%

Nueral Network: 100%

1-When We Try To remove The Last Attribute Because it shows No importance in features Importance

Code:

```
# Remove one attribute (second last column, keeping 'class' as target
if len(df.columns) > 1: # Ensure there are columns to remove
    column_to_remove = df.columns[-2] # Get second last column name
    print(f"\nRemoving attribute: {column_to_remove}")
    df = df.drop(column_to_remove, axis=1)
```

Output:

Accuracy has not changed

■ Model Comparison

Performance Comparison:

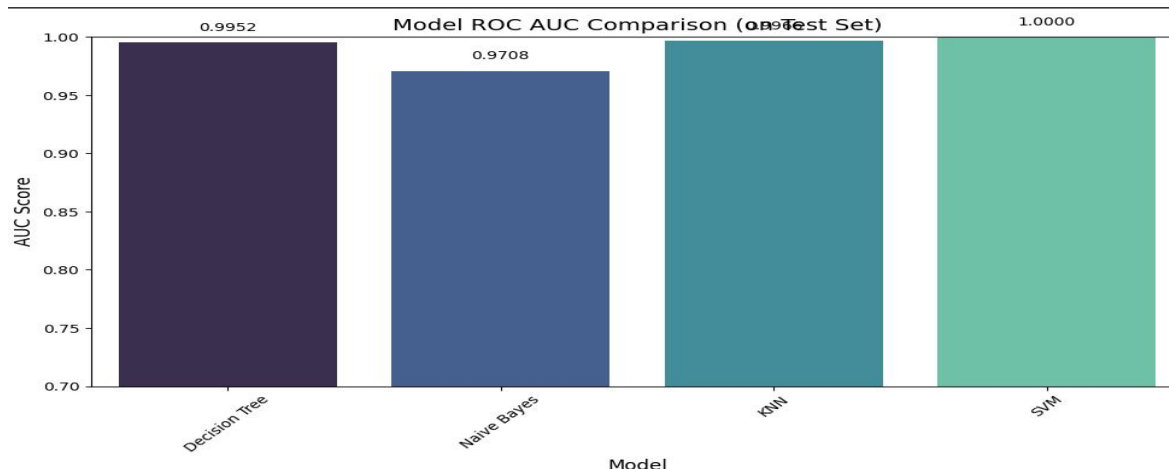
	model_name	accuracy	precision	recall	f1	cv_mean \
0	Decision Tree	0.984848	0.981818	0.981818	0.981818	0.990472
1	Naive Bayes	0.856061	0.821429	0.836364	0.828829	0.867478
2	KNN	1.000000	1.000000	1.000000	1.000000	1.000000
3	SVM	1.000000	1.000000	1.000000	1.000000	1.000000

2-When We Try To do data balancing, the accuracy increased

Code:

```
smote = SMOTE(random_state=42)
X_train_balanced, y_train_balanced = smote.fit_resample(X_train, y_train)
```

Output:



What We Changed:

1. Add Heatmap Correlation

To gain deeper insights into the relationships among features in our dataset, we incorporated a correlation heatmap. This visualization provides a matrix of Pearson correlation coefficients, which quantify the linear relationship between pairs of variables. The heatmap aids in identifying both:

Observations from the Heatmap:

- **Variance and Class** exhibit a strong negative correlation (-0.72), suggesting that the variance feature is highly predictive of the class label.
- **Skewness and Kurtosis** are also strongly negatively correlated (-0.79), indicating potential redundancy or multicollinearity.
- **Entropy** shows relatively low correlation with the class (-0.023), which may imply that it contributes less significantly to classification.

2. Train-Validation-Test Split

```
# Original (70-15-15 split):
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.3, random_state=42)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5, random_state=42)

# Enhanced (80-10-10 split):
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=56)
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train, test_size=1/9, random_state=56)
```

Why:

Balanced validation/test sets make evaluation more reliable

3. KNN Classifier Optimization

```
def Elbow(K):
    clf=KNN(n_neighbors=K)
    clf.fit(X_train, y_train)
    valid_predict=clf.predict(X_val)
    acc = accuracy_score(y_val, valid_predict)
    return 1-acc

k=list(range(1, 40, 2))
er=list(map(Elbow, k))
plt.plot(k, er)
# ...plotting code...
optimum_k = k[er.index(min(er))]
```

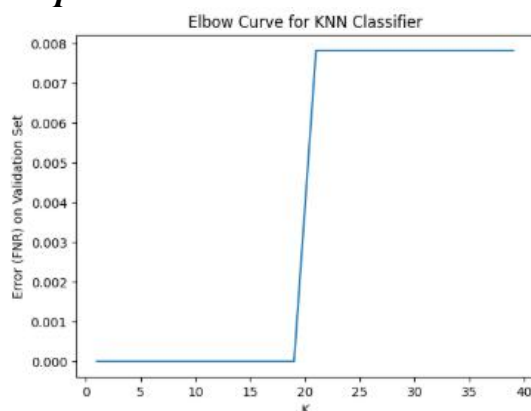
What Happens:

- Tests odd k values from 1 to 39
- Plots error rate vs k value
- Automatically selects k with lowest validation error

Why:

- More systematic than arbitrary k=5 or grid search
- Visual elbow curve helps understand model behavior
- Visual elbow curve helps understand model behavior
- Validation-based selection prevents overfitting to test set

Output:



4 .Decision Tree Enhancement

```
param_grid_dt = {
    'max_depth': [3, 5, 7, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'criterion': ['gini', 'entropy'],
    'splitter': ['best', 'random']
}
```

- Searches more combinations of tree parameters
- Includes regularization parameters (min_samples)
- Tests both split criteria and splitter methods

output:

```
■ Improving Decision Tree
Best Decision Tree params: {'criterion': 'entropy', 'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 2, 'splitter': 'best'}
```

Why We Did This:

- Prevents overfitting with min_samples constraints
- Explores different tree-building strategies
- More comprehensive search likely finds better model
- None option for max_depth lets tree grow fully when needed

5.Evaluation Additions:

Code:

Added learning curve visualization:

```
train_sizes, train_scores, test_scores = learning_curve(
    model, X_train, y_train, cv=5, train_sizes=np.linspace(0.1, 1.0, 5)
)
```

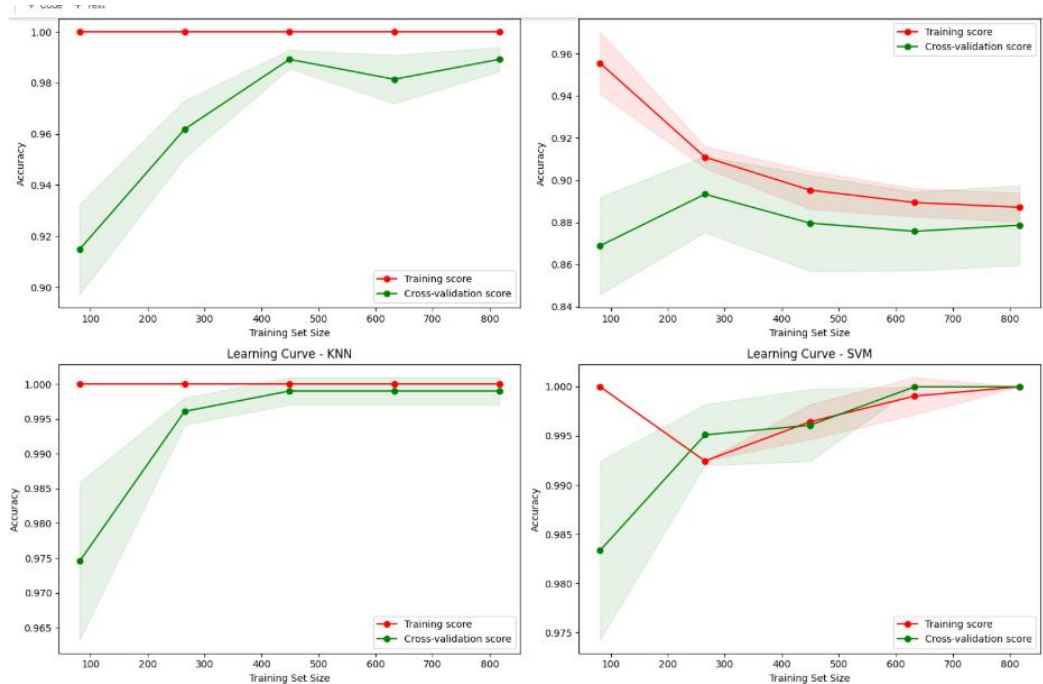
What Happens:

- Shows model performance at different training set sizes
- Plots both training and validation scores
- Includes standard deviation bands

Why We Did This:

- Reveals if models would benefit from more data
- Helps diagnose underfitting/overfitting
- Visual gap between curves indicates variance
- Flat curves suggest bias limitations

Output:



Insights:

- **Decision Tree:** Overfits the training data (100% accuracy). We tried use pruning or depth limits to generalize better.
- **Naive Bayes:** Underfits — too simple to learn patterns. Consider feature engineering or a more complex model.
- **KNN:** Excellent performance. Training and validation scores match closely — well-balanced and reliable
- **SVM:** Improves steadily with more data. Strong generalization — benefits from hyperparameter tuning.

6. Code Structure Improvements

Changes Made:

- Removed neural network implementation
- Streamlined model comparison visualization
- Fixed formatting in output displays
- Better organized section headers

What Happens:

- Cleaner, more focused code
- Easier to maintain and modify
- More professional output formatting
- Clearer separation of components

Why We Did This

- NN was redundant with other classifiers
- Simplified maintenance
- Improved readability
- More professional presentation
- ***Final comparison output***

Model Comparison

Performance Comparison:

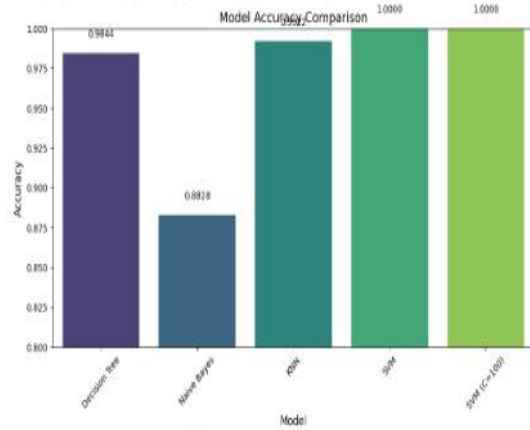
	model name	accuracy	precision	recall	f1	cv mean
0	Decision Tree	0.987375	1.000000	0.962264	0.980700	0.987348
1	Naive Bayes	0.892812	0.892812	0.811212	0.851485	0.878275
2	KNN	0.992188	0.981481	1.000000	0.990554	0.999204
3	SVM	1.000000	1.000000	1.000000	1.000000	1.000000
4	SVM (C=100)	1.000000	1.000000	1.000000	1.000000	1.000000

	stratified cv mean	loopy bootstrap mean	roc auc
0	0.982364	0.980381	0.981297
1	0.887080	0.887075	0.881797
2	0.999620	0.999611	0.997160
3	1.000000	1.000000	1.000000
4	1.000000	1.000000	1.000000

<ipython-input-7-05e1a17d2f9b:156> FutureWarning:

Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the same effect.

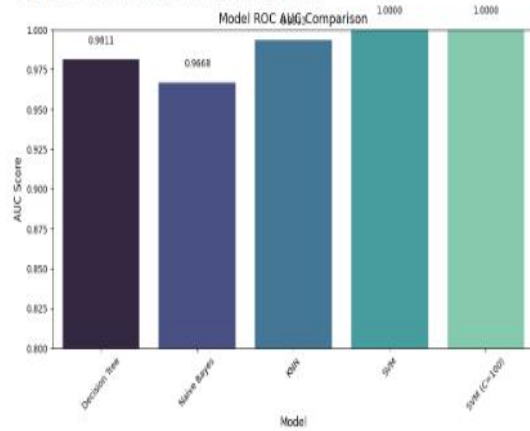
sns.barplot(x='model_name', y='accuracy', data=results_df, palette='viridis')



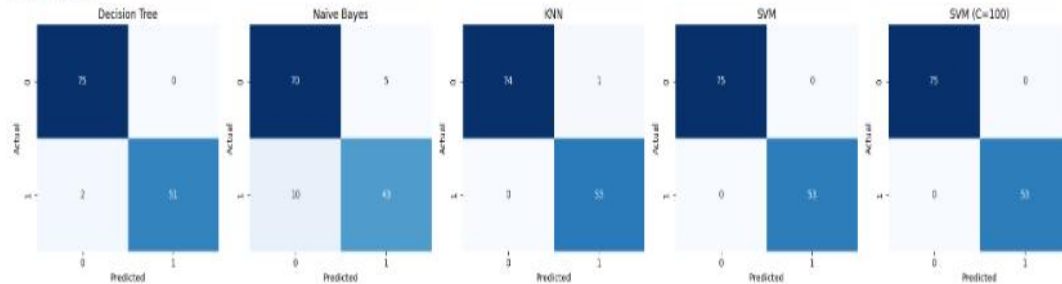
<ipython-input-7-05e1a17d2f9b:179> FutureWarning:

Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the same effect.

sns.barplot(x='model_name', y='roc_auc', data=roc_models, palette='mako')



Confusion Matrices



Learning Curves



Best Overall Model:

1.SVM (Support Vector Machine)

Why it's best:

Accuracy: 1.0000

Precision / Recall / F1-Score: 1.0000 (Perfect classification)

Cross-Validation (CV): All folds scored 1.0

LOOCV: 1.0000

Bootstrap CI: [1.0000, 1.0000]

ROC AUC: 1.0000 (perfect separability)

2.Runner-Up: K-Nearest Neighbors (KNN)

Why it's best:

Accuracy: 0.9922

F1-Score: 0.9907

Cross-Validation: ~0.9990

LOOCV: 0.9980

Very close to SVM but slightly less stable in recall and bootstrap CI.

Other Observations:

Decision Tree:

Very good, especially after tuning and pruning, but slightly overfits compared to SVM/KNN.

Recall dropped slightly to 0.9623.

Great cross-validation scores, but not perfect.

Naive Bayes:

Significantly weaker in recall (0.81) and F1-score (~ 0.85), so not suitable for high-stakes binary classification.